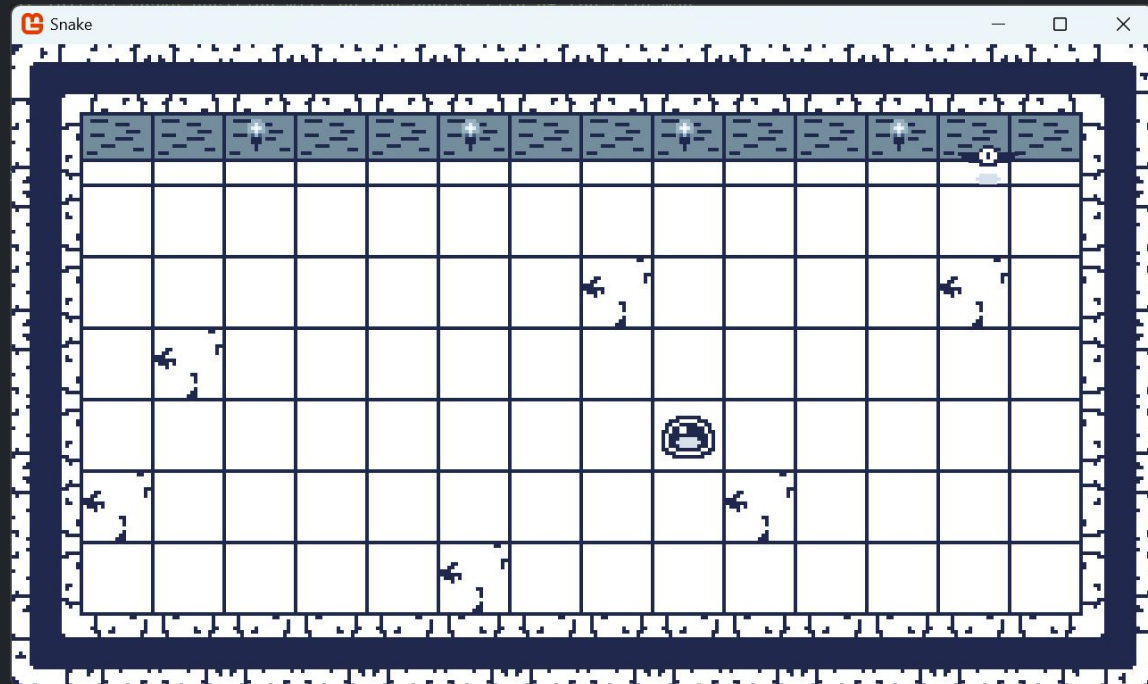


# GMD2

## Lecture 05: Snake

# Today's Goal



# Today's Topics

- Texture Atlases
- Sprites & Animation
- Command Pattern
- Collision Detection
- Tilemaps

## Source Code

- All Source code for today's game can be found here:
  - <https://github.com/Metamate/gmd2-snake>

# Optimizing Texture Rendering



mario1.png



mario2.png



mario3.png



ground1.png



atlas.png

# Texture Atlas



## Texture Region

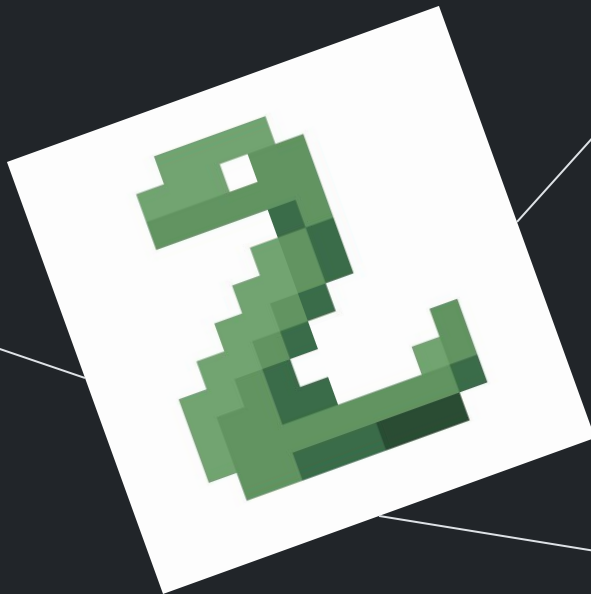


# Texture Atlas Implementation

DEMO!

# Sprites

- Texture (region)
- Position
- Rotation
- Scale
- Color mask
- Origin
- Sprite effects
- Layer depth
- Width
- Height



CenterOrigin()

Draw(Spritebatch, Vector2)

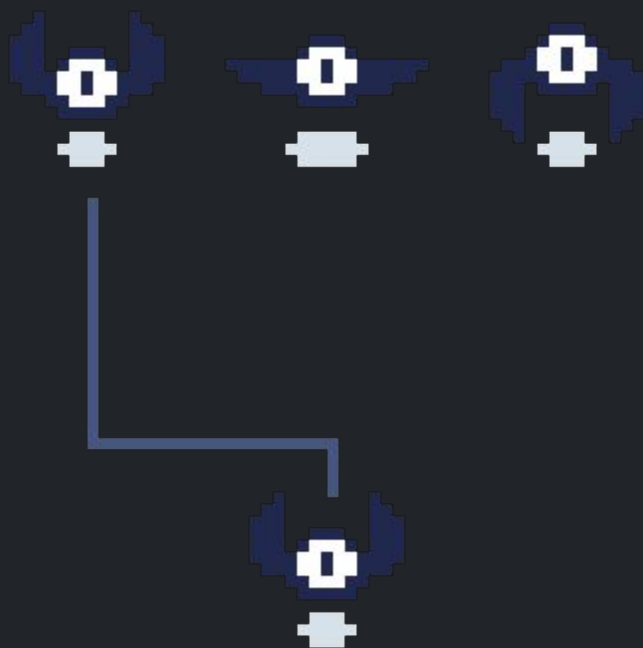
# Sprite Implementation

DEMO!

## Sprite Art



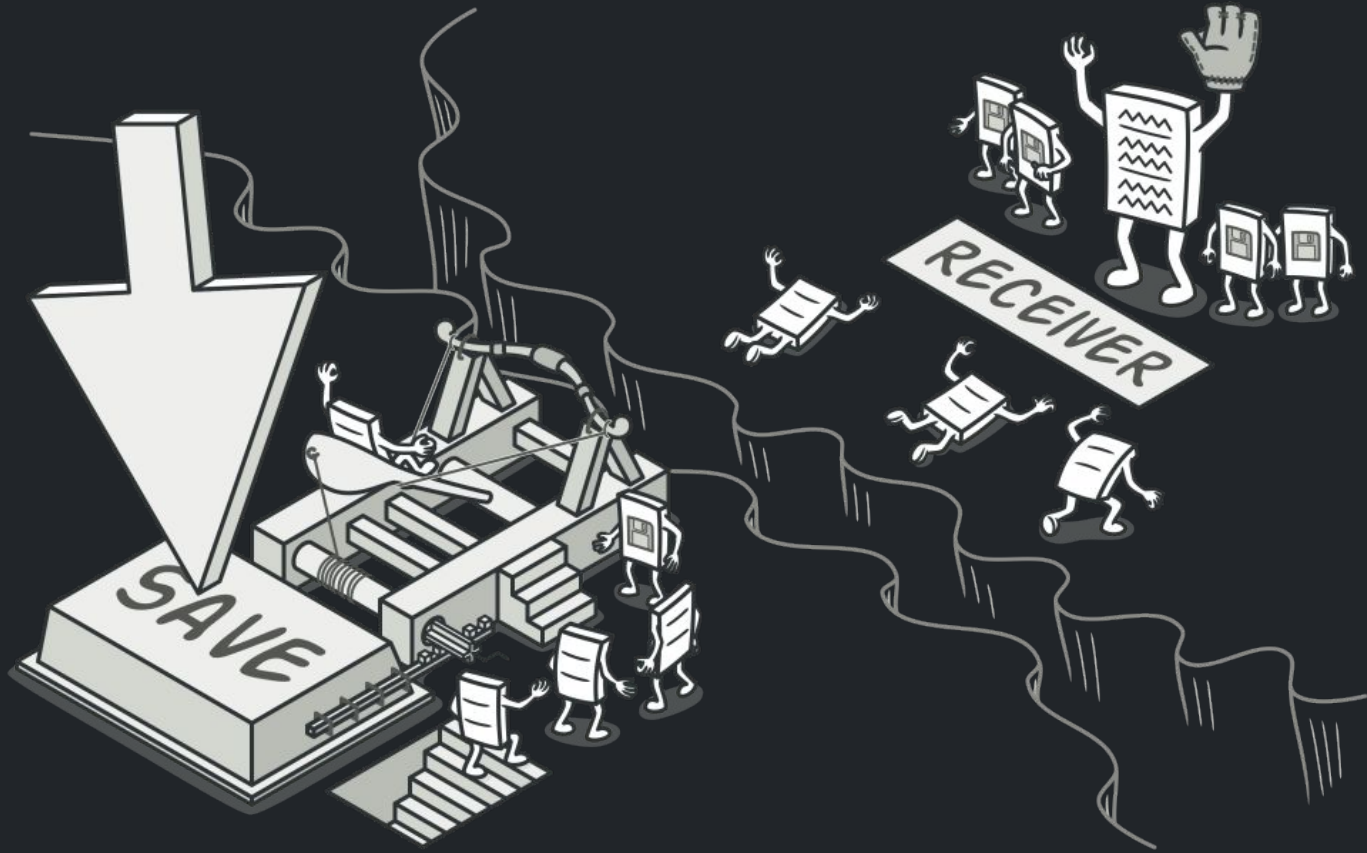
## Animation



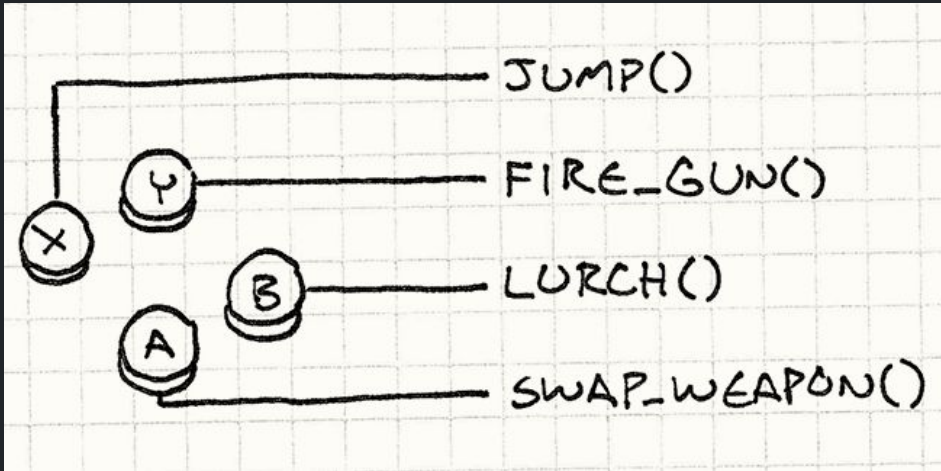
# Animation Implementation

DEMO!

# Command Pattern

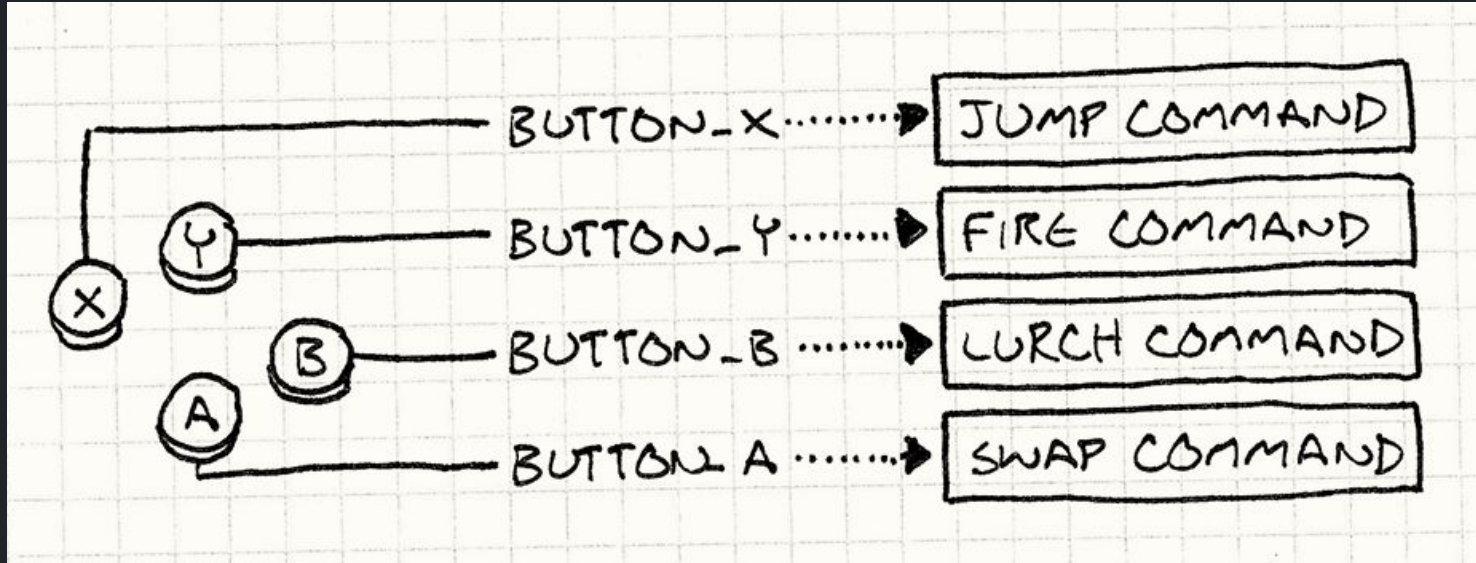


# Command Pattern: Problem



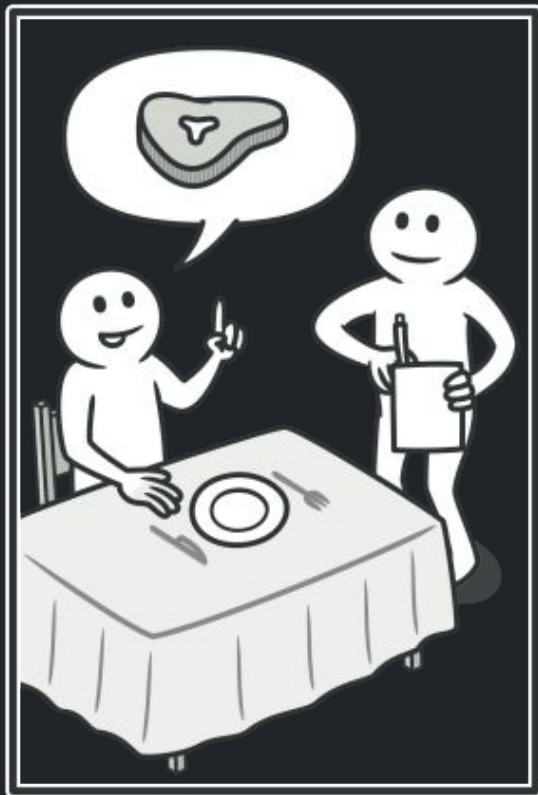
```
public void Update(GameTime gameTime)
{
    if (isPressed(BUTTON_X)) Jump();
    if (isPressed(BUTTON_Y)) FireGun();
    if (isPressed(BUTTON_A)) SwapWeapon();
    if (isPressed(BUTTON_B)) Lurch();
}
```

## Command Pattern: Solution



"A command is a reified method call."

## Command Pattern: Solution



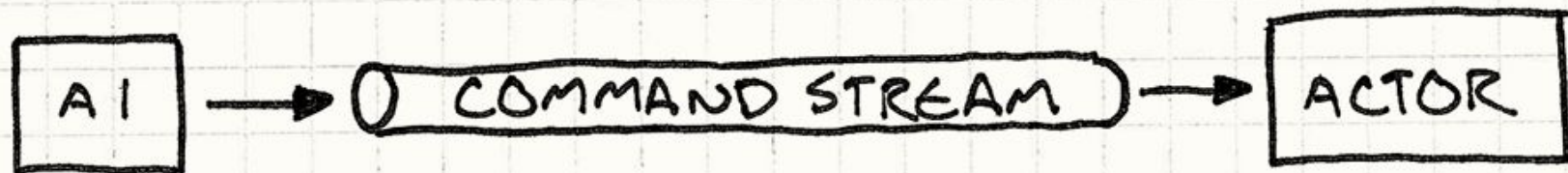
# Command Pattern: Solution

```
public interface ICommand
{
    void Execute();
}
```

```
public class JumpCommand() : ICommand
{
    public void Execute()
    {
        Jump();
    }
}
```

```
public ICommand ButtonX { get; set; } = new JumpCommand();
public ICommand ButtonY { get; set; } = new FireCommand();
public ICommand ButtonB { get; set; } = new LurchCommand();
public ICommand ButtonA { get; set; } = new SwapCommand();
```

```
public void Update(GameTime gameTime)
{
    if (isPressed(BUTTON_X)) ButtonX.Execute();
    if (isPressed(BUTTON_Y)) ButtonY.Execute();
    if (isPressed(BUTTON_A)) ButtonA.Execute();
    if (isPressed(BUTTON_B)) ButtonB.Execute();
}
```



action/request



ICommand  
interface



Command object



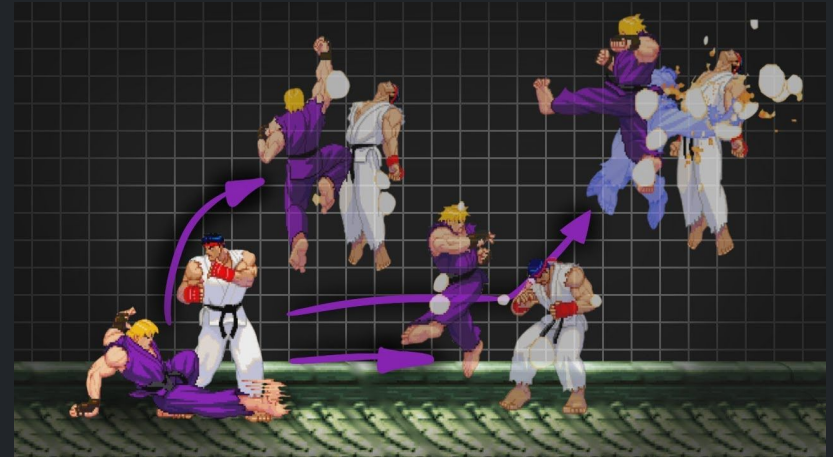
Command stack/buffer



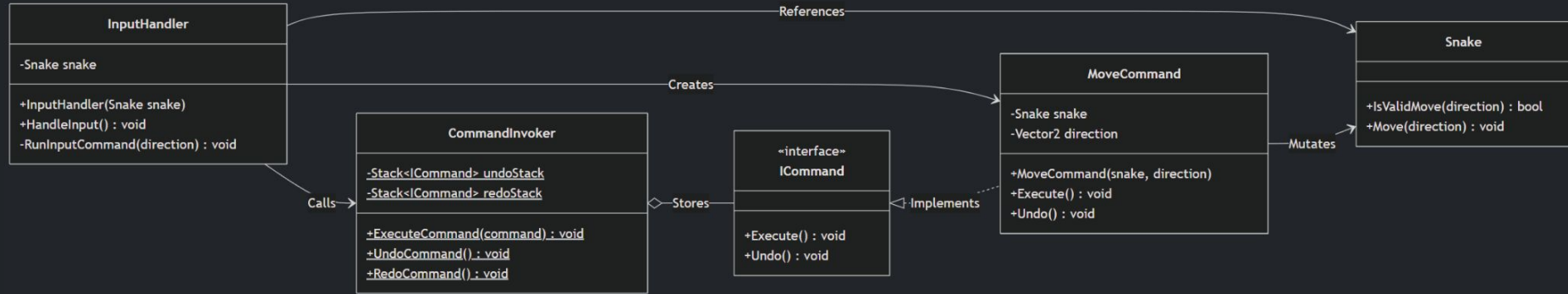
Invoker

# Command Pattern: Usage

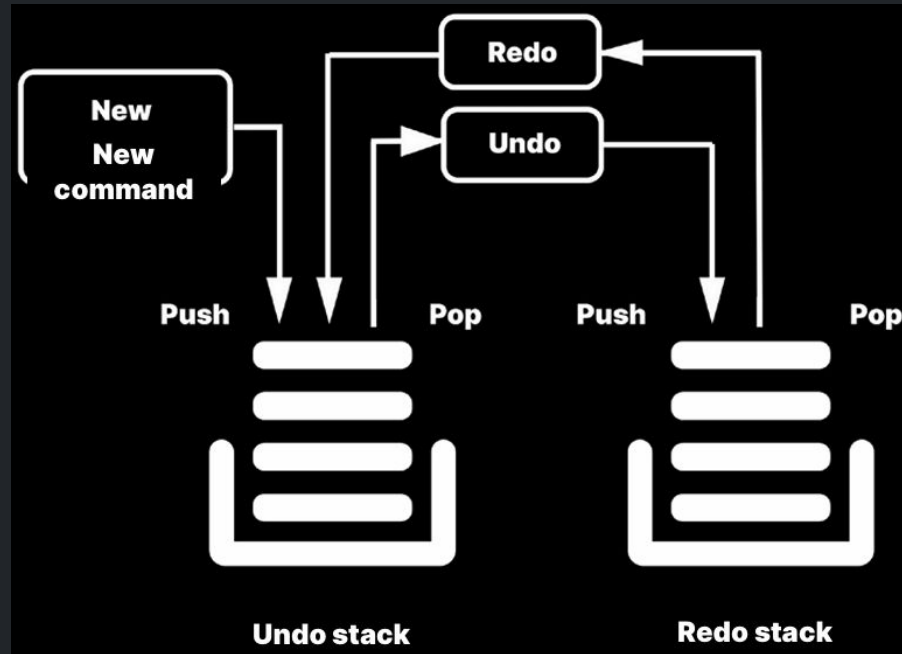
- Input
- Undo
- Redo



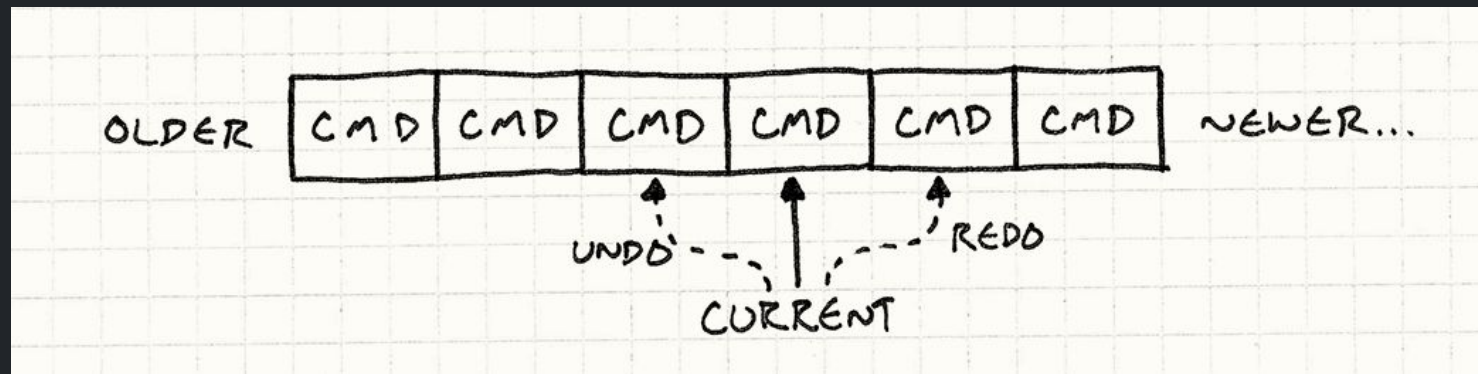
# Adding Undo/Redo Functionality



# Adding Undo/Redo Functionality



## Adding Undo/Redo Functionality

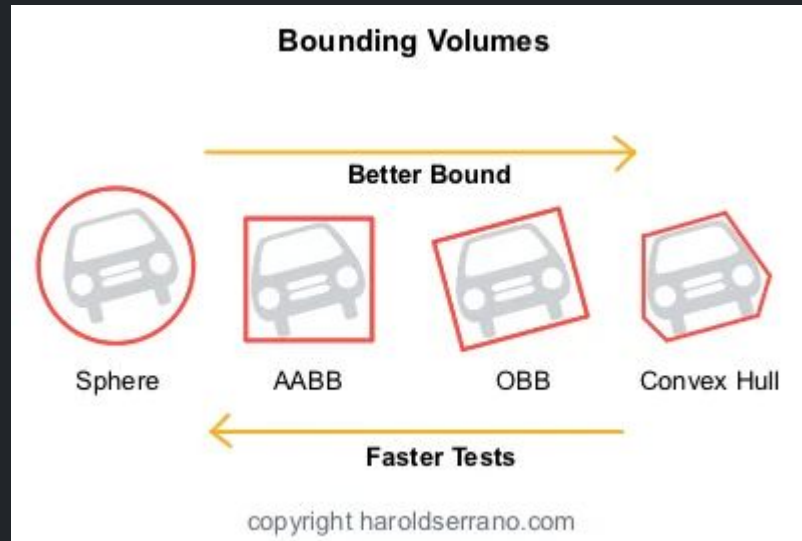


# Command Pattern Implementation

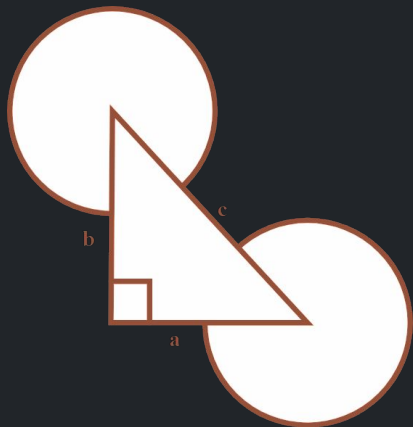
DEMO!

# Collision Detection

- Distance
- Shape-based
  - AABB, Circles
- Complex polygon checks



# Circle Collision Detection



Why not [Vector2.Distance](#)?

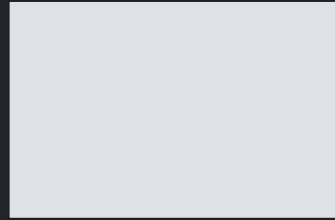
```
Vector2 circle1Position = new(8, 10);
Vector2 circle2Position = new(5, 6);
float circle1Radius = 5;
float circle2Radius = 5;

// c^2 = 25
float distanceSquared = Vector2.DistanceSquared(circle1Position, circle2Position);

// r^2 = 100
float radiiSquared = (circle1Radius + circle2Radius) * (circle1Radius + circle2Radius);

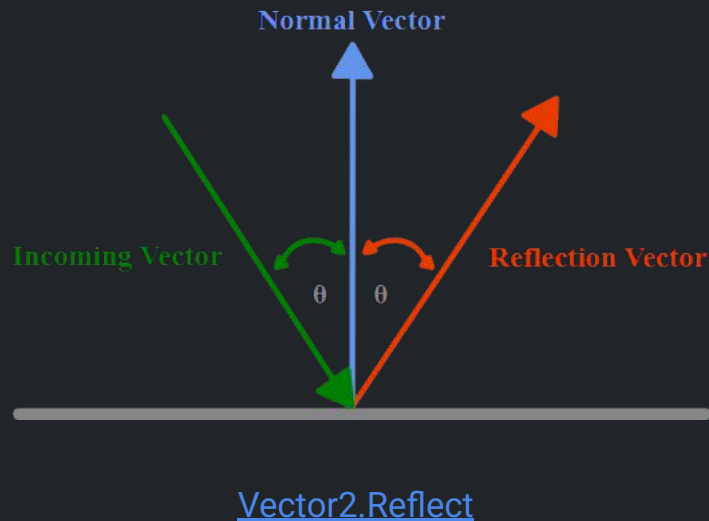
// The circles overlap because 100 is greater than 25.
if (radiiSquared > distanceSquared)
{
}
```

# Collision Detection



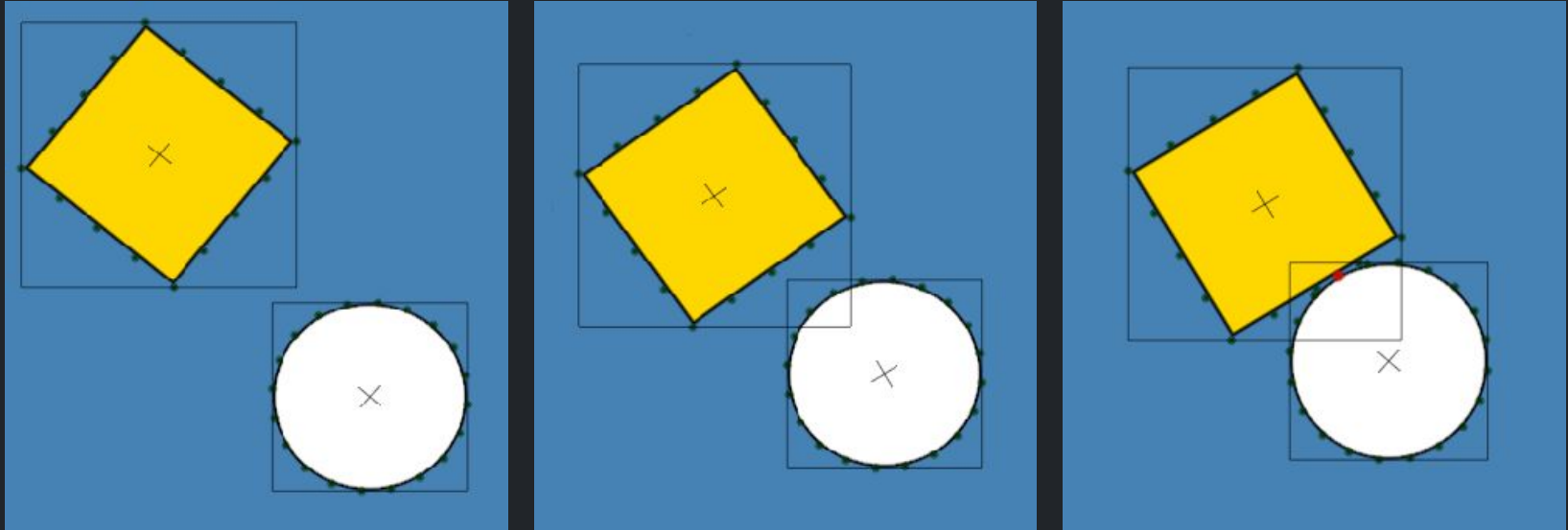
# Collision Response

- Blocking to prevent objects from overlapping
- Triggering events when objects collide
- Bouncing to reflect objects off surfaces



# Optimizing Collision Performance

- Collision Detection is (often) the most expensive operation in a game engine
  - $n * (n - 1) / 2$
  - 100 objects =>  $100 * 99 / 2 = 4950$  collision checks every frame
- Broad phase & Narrow phase



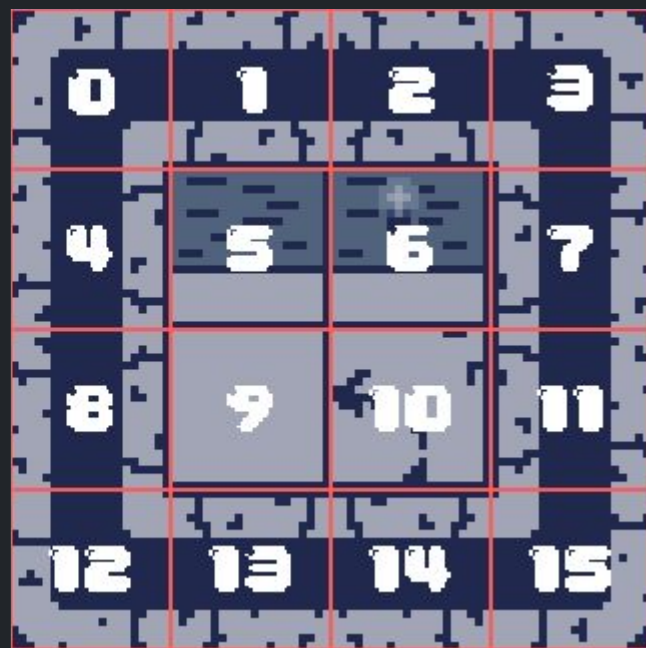
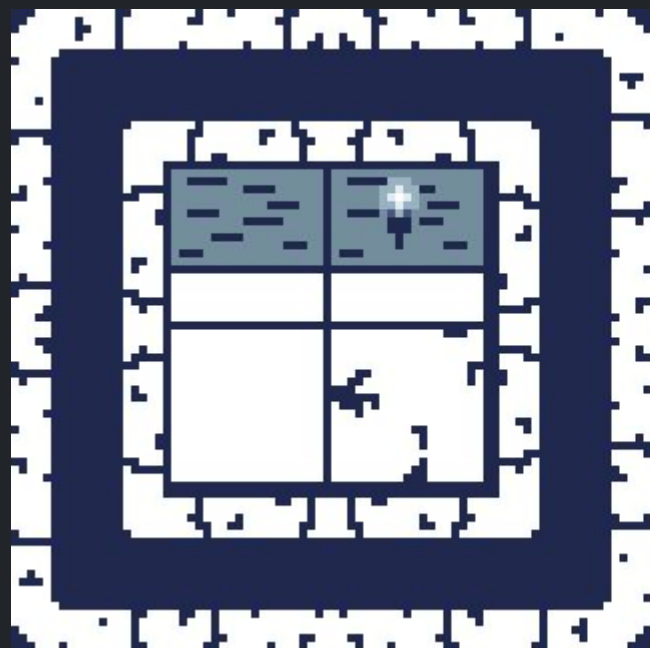
# Refactoring Collisions to GMDCore

- No need to add basic AABB collision detection to our library
- [Rectangle.Intersects\(Rectangle\)](#)
  - Provided by MonoGame
    - performs an an AABB collision check for us
- [Rectangle.Contains\(\)](#)
  - Checks if Rectangle/Vector2/Point is contained within the Rectangle
- No Circles though
  - Add a Circle struct to GMDCore
  - Add an Intersects(Circle) method for other Circles

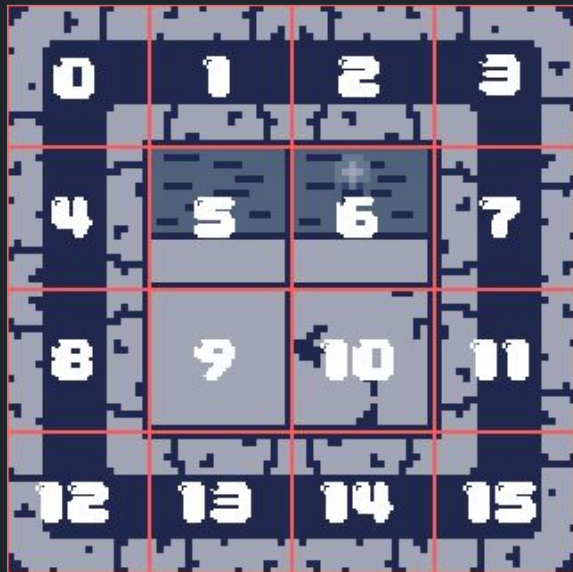
# Collision Implementation

DEMO!

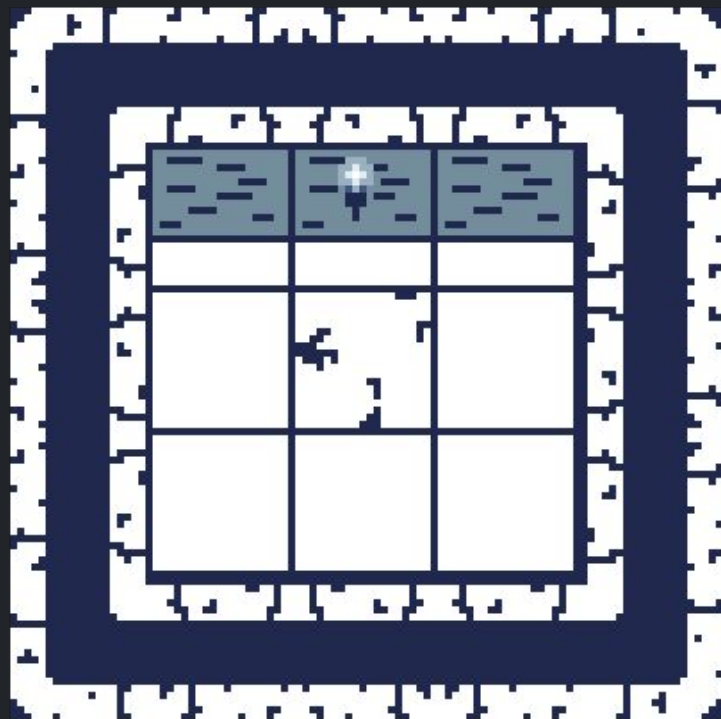
## Tileset



# Tilemap



```
00 01 02 01 03
04 05 06 05 07
08 09 10 09 11
04 09 09 09 07
12 13 14 13 15
```

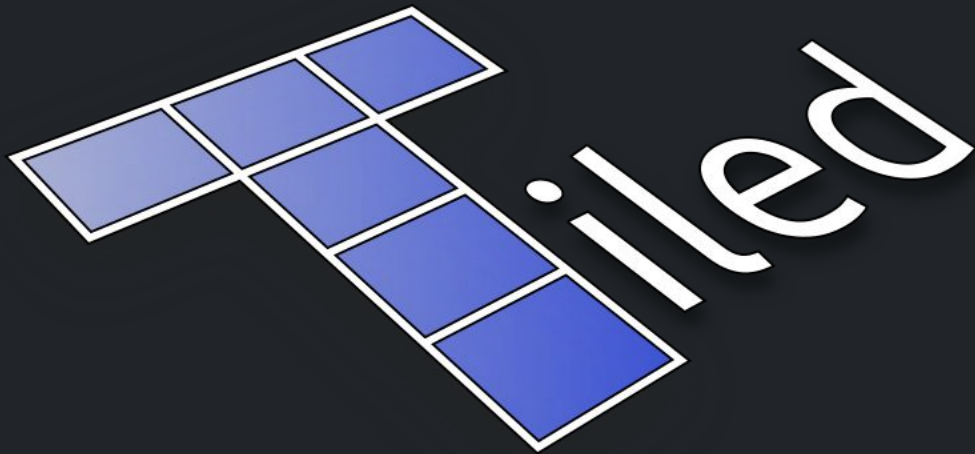


# Tilemap Implementation

DEMO!

# Create Custom Tilemap

- Find (or create) a custom tilemap and apply it to the snake game
  - You will need
    - Texture (atlas)
    - Tilemap definition (xml-file)
- Feel free to change the sprites and animations as well!



# “Extend” Snake Mechanic

- Add a tail to the snake that extends as the game is played
  - E.g. when eating the bat
- Consider what happens when the snake collides with itself
  - E.g. ending the game
- To make the game more action packed, you can consider revamping the movement such that the snake is always moving
  - Currently movement only happens when a key is pressed
  - You will have to update collision response between snake and room bounds
    - Now hitting the walls should end the game
  - This will probably make the current undo/redo unnecessary
- Add a score that increments every time the snake eats the bat
- Take advantage of the command pattern to reverse movement input mappings
  - This could, for example, happen when pressing R or if the snake picks up an item
  - up is down, down is up, left is right, right is left